

Examen Final — Parte II

Cálculo Numérico



Problema 1 [1 punto]

El **coeficiente binomial** se define como

$$\binom{a}{b} = \frac{a!}{b! (a-b)!}, \quad \text{con } n! = 1 \cdot 2 \cdots n.$$

Trabaja en **aritmética de coma flotante** y considera el coeficiente binomial central $\binom{a}{b}$ con $b = \lfloor a/2 \rfloor$ (en Python, `b = a // 2`).

(a) Implementa `binom_factorial(a, b)` con la fórmula anterior. Calculando $\binom{a}{b}$ con $b = \lfloor a/2 \rfloor$ para valores crecientes de a , determina **experimentalmente** el mayor a para el que la función devuelve un resultado finito.

(b) Implementa `formula_producto(a, b)` con la siguiente fórmula

$$\prod_{i=1}^b \frac{a-b+i}{i}.$$

y determina, del mismo modo, el mayor a para el que devuelve un resultado finito.

(c) Comprueba si ambas expresiones son matemáticamente equivalentes y, si lo fueran, explica a qué se debe la diferencia entre el mayor a alcanzable en (a) y en (b).

Problema 2 [2 puntos]

Para la resolución numérica de una ecuación $f(x) = 0$ se considera el **método de Halley**:

$$x_{k+1} = x_k - \frac{2f(x_k)f'(x_k)}{2[f'(x_k)]^2 - f(x_k)f''(x_k)}, \quad k = 0, 1, \dots$$

(a) Implementa el método de Halley en Python:

```
def halley(f, df, d2f, x0, tol=1.e-8, maxit=50, verbose=True):
```

Estima **experimentalmente** el orden de convergencia r y la constante C del método utilizando el error absoluto $|e_k| = |x_k - x^*|$ entre las aproximaciones sucesivas y la raíz real $x^* = 2^{1/3}$ de la función $f(x) = x^3 - 2$ con valor inicial $x_0 = 1.3$. Utiliza una tolerancia de 10^{-8} .

(b) Estudia si el método de Halley puede expresarse como una iteración de **punto fijo** $x_{k+1} = g(x_k)$. Si es así, identifica $g(x)$, obtén la raíz con `scipy.optimize.fixed_point` e indica si el orden de convergencia estimado en (a) concuerda con el que predice la teoría.

Problema 3 [2 puntos]

Como vimos en clase, $f(x) = \frac{1 - \cos x}{x^2}$ sufre **cancelación catastrófica** al evaluarla para x pequeño. Estudiamos su **segunda derivada** en $x = 0.004$ mediante diferencias finitas.

(a) Reescribe $f(x)$ en una forma **algebraicamente equivalente** que **evite** la cancelación. Implementa `f_naive(x)` (original) y `f_estable(x)` (reescrita).

(b) Aproxima $f''(0.004)$ mediante una **diferencia centrada** para $h = 10^{-1}, \dots, 10^{-8}$, usando **ambas** versiones de f . Calcula el **error absoluto** frente al **valor exacto** de $f''(0.004)$ (analítico; puedes usar `sympy`) y dibuja un **gráfico log-log** del error frente a h (guárdalo como `p3_error.png`).

(c) **Interpreta** los resultados.

Se proporciona `starter_p3.py` con un ejemplo de gráfico log-log de `matplotlib`.

Problema 4 [1 punto]

La regla de **Simpson 3/8 compuesta** es una regla de Newton-Cotes (no vista en el temario) que, sobre cada 3 subintervalos de paso h , usa los pesos $\frac{3h}{8}[1, 3, 3, 1]$. Se proporciona su versión **compuesta** (N múltiplo de 3):

```
import numpy as np

def simpson38(f, a, b, N):          # N múltiplo de 3
    assert N % 3 == 0, "N debe ser multiplo de 3"
    x = np.linspace(a, b, N + 1)
    y = f(x)
    h = (b - a) / N
    w = np.ones(N + 1)
    w[1:-1:3] = 3
    w[2:-1:3] = 3
    w[3:-1:3] = 2
    return (3 * h / 8) * np.dot(w, y)
```

Determina **experimentalmente** el **grado de exactitud** de `simpson38` sobre $[0, 1]$.

Se proporciona `starter_p4.py` con la función `simpson38`.